



AI with **Thiru**

L E A R N · B U I L D · D E P L O Y

How to Use MiniMax H3

A clear, practical guide to the open-source MiniMax H3 model. Call it as a hosted API on their site, or run the very same weights offline on your own localhost — with runnable code for both.

Hosted API

vLLM Server

Transformers

Ollama / GGUF

What is MiniMax H3?

MiniMax H3 is an open-source large language model. Open source here means the trained weights are published, not just an endpoint. That single fact is what makes this guide possible: you are never locked to one provider.

You can send a request to MiniMax's hosted API and get an answer in about a minute of setup. Or you can download the same weights and serve them from your own machine, where nothing leaves your network and there is no per-token bill. Same model, four different doors.

The four routes

Hosted

Their site. No GPU, no download. Pay per token.

vLLM

Localhost. The fast, production way to self-host.

Transformers

Localhost. Plain Python, no server to run.

Ollama

Localhost. Quantized, fits modest hardware.

Read the routes like this: the hosted API is the fastest way to a first answer. vLLM is what you run on your own hardware when other people depend on it. Transformers is for scripts and experiments. Ollama is for a laptop or a single small GPU.

A simple analogy

The hosted API is like ordering from a restaurant. Someone else owns the kitchen, keeps it running, and hands you a bill. Running it on localhost is buying the ingredients and cooking at home: more setup on day one, but full control of the recipe, no bill per plate, and your data never leaves the building.

The Four at a Glance

Route	Where it runs	Hardware	Setup effort
Hosted API	MiniMax's servers	None	An API key
vLLM	Your localhost	Server-class GPU(s)	One serve command
Transformers	Your localhost	One capable GPU	pip install only
Ollama / GGUF	Your localhost	CPU or small GPU	One binary

Setup: install once

Everything in this guide runs on the standard Python AI stack. Install it once and every later page just works.

TERMINAL SETUP.SH

```
# Client + local inference essentials
pip install openai transformers accelerate torch huggingface_hub

# Only if you plan to serve the model yourself (page 5)
pip install vllm
```

Two values to fill in. Every script below starts with a `MODEL_ID` and a `BASE_URL`. Copy those two strings from the official MiniMax model card and API docs for the H3 release you are using, then leave the rest of the code untouched — the workflow does not change between versions.

Before you download anything

- Check the model card for the parameter count and context length, and confirm your GPU has the memory for it in bf16. Roughly: bf16 needs about 2 GB of VRAM per billion active parameters, plus headroom for the KV cache.
- If that number is larger than your GPU, do not give up — go to the quantized routes on pages 5 and 7 instead.
- Read the licence on the model card before shipping anything commercial.

The Hosted API

What it is: a request to MiniMax's own servers. They own the GPUs, the scaling, and the uptime. You send messages and get completions back, billed per token. The endpoint is OpenAI-compatible, so the official `openai` client works unchanged — only the base URL differs.

Use it when you are prototyping, you have no GPU, or your traffic is spiky enough that renting beats owning.

- 1 Create an account on the MiniMax open platform and open the API keys page.
- 2 Generate a key and copy it once — it is usually shown only at creation time.
- 3 Export it as an environment variable — `export MINIMAX_API_KEY="your_key_here"` — and never paste a key into source code you will commit.

PYTHON HOSTED_API.PY

```
import os
from openai import OpenAI

MODEL_ID = "MiniMax-H3"          # exact id from the API docs
BASE_URL = "https://api.minimax.io/v1" # confirm in the docs

client = OpenAI(api_key=os.environ["MINIMAX_API_KEY"], base_url=BASE_URL)

response = client.chat.completions.create(
    model=MODEL_ID,
    messages=[
        {"role": "system", "content": "You are a concise technical assistant."},
        {"role": "user", "content": "Explain vector databases in three sentences."},
    ],
    temperature=0.7,
    max_tokens=512,
)

print(response.choices[0].message.content)
```

PYTHON STREAMING.PY

```
# Same call, streamed token by token for a responsive UI
stream = client.chat.completions.create(
    model=MODEL_ID, messages=[{"role": "user", "content": "Write a haiku about GPUs."}],
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)
```

ROUTE TWO

Localhost with vLLM

What it is: vLLM downloads the open weights and serves them from your own machine behind an OpenAI-compatible HTTP server. It batches requests, manages the KV cache efficiently, and is the standard way to self-host a model in production.

Use it when you own a capable GPU, need real throughput, or the data must never leave your network. Once it is running, the client code is identical to Route One with the base URL pointed at `localhost`.

TERMINAL SERVE_LOCAL.SH

```
# 1) Pull the open weights (a few minutes to a few hours, by size)
hf download MiniMaxAI/MiniMax-H3 --local-dir ./minimax-h3

# 2) Serve them on localhost:8000
vllm serve ./minimax-h3 \
  --served-model-name minimax-h3 \
  --host 0.0.0.0 --port 8000 \
  --tensor-parallel-size 1 \
  --max-model-len 32768 \
  --trust-remote-code

# Low on VRAM? Add 4-bit loading and shrink the context:
# --quantization bitsandbytes --max-model-len 8192
```

Wait for the line `Application startup complete`, then point any OpenAI client at the local port. The API key is required by the protocol but ignored, so any placeholder string works.

PYTHON LOCAL_VLLM_CLIENT.PY

```
from openai import OpenAI

client = OpenAI(api_key="EMPTY", base_url="http://localhost:8000/v1")

response = client.chat.completions.create(
    model="minimax-h3", # must match --served-model-name
    messages=[{"role": "user", "content": "Summarise this guide in one line."}],
    temperature=0.7,
    max_tokens=256,
)

print(response.choices[0].message.content)
```

The knob that matters most. `--tensor-parallel-size` should equal the number of GPUs you are splitting the model across. `--max-model-len` is the second lever: halving it frees a large amount of KV-cache memory and is usually the quickest fix for an out-of-memory crash.

Localhost with Transformers

What it is: loading the weights straight into Python with Hugging Face Transformers and calling `generate()` yourself. No server, no HTTP, no extra moving parts — just the model object in your process.

Use it when you are writing a one-off script, batch-processing a file, or poking at the model's internals. It is the simplest route to reason about, and the slowest under concurrent load, which is exactly why Route Two exists.

PYTHON LOCAL_TRANSFORMERS.PY

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "MiniMaxAI/MiniMax-H3" # or a local folder path

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.bfloat16, # half the memory of float32
    device_map="auto",         # spread across whatever GPUs exist
    trust_remote_code=True,
)

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Give me three ideas for a weekend AI project."},
]

# apply_chat_template inserts the special tokens this model expects
inputs = tokenizer.apply_chat_template(
    messages, add_generation_prompt=True, return_tensors="pt"
).to(model.device)

with torch.no_grad():
    output = model.generate(
        inputs, max_new_tokens=512, temperature=0.7, top_p=0.9, do_sample=True
    )

# Slice off the prompt so only the reply is decoded
reply = tokenizer.decode(output[0][inputs.shape[-1]:], skip_special_tokens=True)
print(reply)
```

Always use the chat template. Feeding raw text to a chat model is the single most common cause of rambling or truncated answers. `apply_chat_template()` reads the format from the model's own config, so you never have to hand-write role markers.

Localhost on Modest Hardware

What it is: running a quantized copy of the weights — a GGUF build — through Ollama or llama.cpp. Quantizing compresses each weight to roughly 4 bits, which cuts memory use dramatically for a small quality cost, and lets the model spill onto CPU RAM when the GPU runs out.

Use it when you are on a laptop, a single consumer GPU, or an Apple Silicon Mac. This is the route that turns "I need a server" into "it runs on my machine".

TERMINAL OLLAMA_RUN.SH

```
# Install Ollama once, then pull a GGUF build of the model
ollama pull minimax-h3

# Chat with it straight from the terminal
ollama run minimax-h3 "Explain quantization like I am five."

# Ollama also exposes an OpenAI-compatible server on port 11434
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "minimax-h3", "messages": [{"role": "user", "content": "Hello!"}]}'
```

Because that server speaks the same dialect, your Python code is once again the same three lines — only the port changes.

PYTHON LOCAL_OLLAMA_CLIENT.PY

```
from openai import OpenAI

client = OpenAI(api_key="ollama", base_url="http://localhost:11434/v1")

response = client.chat.completions.create(
    model="minimax-h3",
    messages=[{"role": "user", "content": "Draft a polite follow-up email."}],
)

print(response.choices[0].message.content)
```

One dependency to check. This route needs a published GGUF conversion of H3. If `ollama pull` reports that the model is unknown, search the Hugging Face hub for a GGUF build and load it with a two-line `Modelfile`, or fall back to Route Two with `--quantization`.

Side-by-Side Comparison

Route	Runs on	Cost model	Data leaves you	Best for
Hosted API	Their servers	Per token	Yes	Fastest start
vLLM	Your GPUs	Hardware only	No	Production self-hosting
Transformers	Your GPU	Hardware only	No	Scripts and research
Ollama / GGUF	Laptop or CPU	Free	No	Small machines

The whole guide in one line. The hosted API rents someone else's GPUs; vLLM, Transformers, and Ollama all run the same open weights on yours, differing only in how much speed, simplicity, or memory you are optimising for.

How to choose

- **Just exploring?** Start with the hosted API. Nothing to install, and you learn the message format you will reuse everywhere else.
- **Privacy or volume matters?** Move to vLLM. The client code you already wrote keeps working — you only swap the base URL.
- **Writing a script, not a service?** Transformers is fewer moving parts.
- **No server-class GPU?** Ollama with a quantized build.

When it goes wrong

Symptom	Usual fix
CUDA out of memory	Lower <code>--max-model-len</code> , load in 4-bit, or move to Route Four
Model not found (404)	The <code>model</code> field must match <code>--served-model-name</code>
401 Unauthorized	The key is missing from the environment, or the base URL is the wrong region
Rambling or cut-off replies	You skipped the chat template, or <code>max_tokens</code> is too low
Custom code error on load	Add <code>trust_remote_code=True</code> and update Transformers

RECAP

The Four, in One View

A final summary you can return to. Each route in one sentence, in the order you would usually reach for them.

Hosted

Your first five minutes

One API key and an OpenAI-compatible call. No hardware, no download, and the exact message format you will reuse on every other route.

vLLM

When it has to be yours

The same weights served from localhost with real throughput. Change one base URL and your existing client code keeps working, offline.

Transformers

When you want the model in-process

Load, tokenize, generate. Fewest moving parts, ideal for scripts, batch jobs, and poking at how the model actually behaves.

Ollama

When the hardware is modest

A 4-bit build that runs on a laptop or a single consumer GPU, still speaking the same OpenAI dialect on port 11434.



Thanks for reading

AI WITH THIRU

If this guide helped, subscribe to **AI with Thiru** for more hands-on AI, machine learning, and LLM tutorials. Try the hosted call first, then run the same prompt on localhost and compare the speed, the cost, and the answer.